Review 1 Work Plan: Stack Overflow in the AI Era

1. Overview

Review 1 carries 20 marks and covers Units 1 and 2. The required deliverables are a self-collected dataset, cleaning and preprocessing, exploratory analysis, feature engineering, a predictive model with evaluation, and a 10-minute presentation. Each member leads one area mapped to rubric marks, and every member also presents, reviews another member's work and keeps the GitHub board current.

Review 1 question: whether a Stack Overflow question receives an accepted answer within 7 days of posting. Main target: `target_resolved_7d`. Since acceptance depends on the asker returning, a secondary target, `target_answered_7d` (any answer within 7 days), serves as a robustness check.

Roles and rubric ownership

Member	Lead area	Rubric components owned	Marks covered
Member 1	Data collection and dataset documentation	Dataset description (shared), data source documentation	1 of 2
Member 2	Data enrichment, cleaning and preprocessing	Data cleaning and preprocessing	3
Member 3	Problem framing, EDA and slide deck	Problem statement and objective (shared), EDA and visualisation	1 of 2, plus 3
Member 4	Feature engineering, dimensionality reduction, baseline models	Feature engineering, model development (shared)	2, plus 2 of 4
Member 5	Advanced models, evaluation, business findings	Model development (shared), model evaluation and interpretation	2 of 4, plus 3
Everyone	Own presentation part and viva answers	Presentation and individual contribution	3

How the load stays equal

- Member 1 carries the heaviest early technical work (API collection) and later assembles the final notebook.
- Member 2 builds the second half of collection (answers and users) plus all cleaning.
- Member 3 writes the business framing, owns the largest set of charts, and builds the slides.
- Members 4 and 5 split modelling: Member 4 builds the features, split and baselines; Member 5 builds the stronger models and all evaluation.
- Everyone has a preparation task in early phases so nobody waits idle for data.

Fixed project decisions (from the project plan)

- Source: Stack Exchange API v2.3, site stackoverflow, accessed with the project's registered key.
- Tags: python, javascript, java, sql, pandas, reactjs, android, excel, r, docker, powerbi, one newer tag agreed in Phase 0, and `openai-api` as an AI contrast tag (its history starts in 2023, so it serves volume and topic contrast only).

- Window: January 2019 to August 2026.
- Sampling: before January 2025, all questions on 2 random weekdays and 1 random weekend day per tag-month (seed 42), each question carrying a sampling weight so monthly figures reflect the month's real mix of weekdays and weekends. Tags with low monthly volume in the test pull are collected in full. From January 2025, all questions are collected.
- Measurement grain: question volume is weekly (complete counts); resolution rate and time to answer are monthly (weighted), since sampled days cannot support weekly rates.
- Targets: main `target_resolved_7d` (accepted answer within 7 days); secondary `target_answered_7d` (any answer within 7 days).
- Split: train Jan 2019 to Dec 2024, validation Jan to Jun 2025, test Jul 2025 to Jul 2026, plus rolling-origin evaluation by half-year from 2021 to 2026.
- Leakage rule: score, views, answer count, is_answered, closed status, comments and current reputation are excluded as features. `post_ai` and `first_question` are not model features. Tag volume features use only weeks before the posting date.
- Licence: Stack Overflow content is CC BY-SA 4.0. Slides and outputs show question titles only, each linked to the original question, with a licence notice; raw text is not published.

2. GitHub Projects setup and task rules

Every piece of work is a GitHub issue on one project board, owned by one member, with a due date and linked evidence. The board's history is how examiners verify individual contribution, so it must be updated steadily, never in a rush before the review.

Repository layout (created in Phase 0)

Path	Contents	Owner
`README.md`	Project summary, how to run, folder guide	Member 1
`requirements.txt`	Pinned Python packages	Member 1
`src/collect.py` , `src/enrich.py`	Collection and enrichment scripts	Members 1 and 2

Path	Contents	Owner
`data/raw/`	Untouched API responses (JSON)	Members 1 and 2
`data/processed/`	Cleaned tables (Parquet or CSV)	Member 2
`logs/collection_log.csv`	One row per API call batch	Members 1 and 2
`docs/data_dictionary.md`, `docs/dataset_documentation.md`	Dataset documentation	Member 1
`notebooks/01_collection.ipynb`	Collection walkthrough and checks	Member 1
`notebooks/02_cleaning.ipynb`	Cleaning and preprocessing	Member 2
`notebooks/03_eda.ipynb`	EDA and visualisations	Member 3
`notebooks/04_features.ipynb`	Features, TF-IDF + SVD, split	Member 4
`notebooks/05_models.ipynb`	Baselines and advanced models	Members 4 and 5
`notebooks/06_evaluation.ipynb`	Evaluation, SHAP, error analysis	Member 5
`notebooks/review1_main.ipynb`	Final combined notebook for submission	Member 1
`reports/review1/`	Slides, findings page, contribution summaries	Member 3

Raw data is too large for Git, so raw files are kept on a shared drive and only a small sample plus the log is committed. The API key is kept in a `.env` file listed in `.gitignore` and is never committed.

Board setup (GitHub Projects, Board view)

- Status field options, in order: Backlog, To Do, In Progress, Review/Testing, Completed.
- Custom fields: Due date (date), Phase (Phase 0 to Phase 5), Rubric component (single select), Reviewer (text), Evidence (text, a link).
- Labels: data, cleaning, eda, features, model, evaluation, docs, presentation, review.

- Extra views: a table grouped by assignee (for contribution summaries) and a table grouped by phase.

Task card template (use for every issue)

```
Title: [M2] Strip HTML and extract code blocks from question bodies

What: One or two sentences on the outcome.
Steps:
1. ...
2. ...
Done when: Clear, checkable condition.
Evidence: Link to commit, PR, notebook section, chart, doc or slide.
Depends on: #issue numbers.
Reviewer: Member X
```

Update rules

1. At the start of each phase, each member creates that phase's tasks in Backlog, using the titles in the member sections.
2. A task moves to To Do when it becomes workable, and its due date is set at that point.
3. A task moves to In Progress when work starts, with a short start comment (for example, "Started: pulling python tag for 2019").
4. During work, progress comments are posted at least twice a week, and commits reference the issue number (`#23 add backoff handling`).
5. On completion, the owner opens a pull request with "Closes #23", adds the evidence link to the Evidence field, and moves the task to Review/Testing.
6. The reviewer checks the work, comments on what was verified, approves the pull request, and moves the task to Completed after merging.
7. Tasks are not created and closed in one sitting, not bulk-moved before a review, and not edited by another member without a comment.

Branch and commit conventions

- Branch per task: `m1/collect-questions`, `m3/eda-trend-charts`.
- Commit messages start with the issue number and a verb: `#31 add PR-AUC to evaluation`.
- Every member commits their own work from their own GitHub account.

3. Phases at a glance

The work runs in six phases that follow the data: each member leads one phase and has preparation or support tasks in every other phase, so nobody waits idle.

Each member leads one phase and works in every phase

Highlighted cell = that member's lead phase; task IDs are in each member's section

	Phase 0 Setup	Phase 1 Collection	Phase 2 Cleaning	Phase 3 EDA + features	Phase 4 Models	Phase 5 Presentation
Member 1	Repo, board, API key	**Counts and questions**	Cross-check, data docs	Coverage charts, rerun	Main notebook skeleton	Final run, slides
Member 2	Environment, enrich design	Answers and user dates	**Merge, clean, target**	Monthly tables	Sampling robustness	Section, slides
Member 3	Problem statement	EDA plan, deck template	Early EDA on sample	**EDA charts and tests**	Findings, fill deck	Rehearsals, slides
Member 4	Split plan, feature spec	Feature functions	Split module and checks	**Features, TF-IDF + SVD**	Baselines, feature gain	Section, slides
Member 5	Eval plan, model setup	Evaluation module	Error analysis plan	Leakage check smoke test	**RF, boosting, eval, SHAP**	Section, slides

who does what in each phase · 5 members × 6 phases

Each row shows one member's order of work; each column shows the team's work in that phase.

A phase is finished when

Phase	Exit condition
Phase 0: Setup	Repo, board, environment and API key work; every member has created their Phase 1 tasks
Phase 1: Collection	Weekly counts, questions, answers and users collected; collection log complete
Phase 2: Cleaning	`questions_clean_v1.parquet` saved with the before/after table; documentation drafted
Phase 3: EDA and features	EDA notebook with takeaways; feature matrix and pipeline saved; leakage check signed off

Phase	Exit condition
Phase 4: Models	Validation comparison done; test evaluated once; SHAP, error analysis and findings page written
Phase 5: Presentation	Main notebook runs top to bottom; deck rehearsed; summaries submitted; board clean

4. Member 1: data collection and dataset documentation

Member 1 builds the data foundation the rest of the team depends on, documents it for the dataset marks, and assembles the final Review 1 notebook. Each bold line is the GitHub issue title for that task.

Phase 0: Setup

1. **[M1-01] Repository, folder structure and README skeleton**
   - Scope: the repository with every folder from Section 2, a `.gitignore` covering `.env` and `data/raw/`, and a README with the project title, a one-paragraph summary and a folder guide.
   - Evidence: repository link and first commit. Complete when all members can clone it.
2. **[M1-02] GitHub Projects board setup**
   - Scope: the board with the five Status options, the custom fields and labels from Section 2, and the assignee and phase views, with all members invited.
   - Evidence: board link. Complete when every member has created one test task.
3. **[M1-03] Stack Apps key and first API call**
   - Scope: app registration on Stack Apps, the key stored in `.env`, and one test call to `/2.3/questions` for one tag and one day, confirming that `quota_max` shows 10,000.
   - Evidence: notebook cell output. Complete when the call works and the key setup steps are in the README.

Phase 1: Data collection (lead phase)

4. **[M1-04] Core API request function**
   - Scope: a request function in `src/collect.py` that adds the key, waits at least 0.5 seconds between calls, honours the `backoff` field, retries errors with increasing waits, saves every raw response as JSON, and appends a row to `logs/collection_log.csv`.

- Also: a custom filter that includes `body`, with the filter string saved in the config.
- Evidence: pull request. Complete when a forced error is retried and logged correctly.

5. **[M1-05] Test pull and volume check**
   - Scope: one month of python questions and weekly counts, reporting rows, fields present, calls used, and questions per weekday and per weekend day.
   - Also: monthly counts for every tag, with the low-volume tags listed for full collection (cut-off agreed by the team in the issue comments), and a check that the sampling rule gives 150,000 to 250,000 questions, with a proposed change if it does not.
   - Evidence: test notebook section and an issue comment with the numbers.
6. **[M1-06] Weekly question counts**
   - Scope: counts from `/questions` with `filter=total` for each tag (including `openai-api`) and each week from January 2019 to August 2026, plus the weekly site-wide total with no tag filter, to show the selected tags' share of all questions.
   - Output: `data/processed/weekly_counts.csv` (tag, week_start, count; tag = "ALL" for the site total).
   - Evidence: file and log rows. Complete when there are no missing weeks, or missing weeks are listed.
7. **[M1-07] Question records with stratified sampling**
   - Scope: before 2025, all questions on 2 random weekdays and 1 random weekend day per tag-month (seed 42); full-collection tags and all months from January 2025 collected in full.
   - Sampling weight on every question: the number of days of that type (weekday or weekend) in the month divided by the number sampled; fully collected questions have weight 1.
   - Output: a sampling manifest (tag, month, sampled dates, weights) and `data/processed/questions_raw.parquet`. Files are handed to Member 2 tag by tag as each finishes, with a comment on Member 2's enrichment task.
   - Evidence: file, manifest, log, and row counts per tag in the issue.
8. **[M1-08] Collection summary**
   - Scope: calls made, items collected, days of quota used, and any failures with their fixes, taken from the log.
   - Evidence: summary table in `01_collection.ipynb`.

Phase 2: Documentation and validation

9. **[M1-09] Cross-check of totals and headline numbers with Stack Exchange Data Explorer**
    - Scope: monthly question counts for 3 tags queried on Data Explorer and compared with the weekly counts summed by month, with the percentage difference and an explanation of any gap.
    - Also: verification of the headline figures planned for slides (yearly totals for 2022 and 2025, July 2026, and the March 2014 peak). Published sources disagree slightly, so slides use only these verified numbers.
    - Evidence: comparison table and saved SQL queries in `01_collection.ipynb`.
10. **[M1-10] Data dictionary**
    - Scope: `docs/data_dictionary.md` with every collected field (name, type, endpoint, meaning) and every derived field (formula, owner), including `sampling_weight`, both targets and the lagged tag-volume features. Derived-field details come from Members 2 and 4 through issue comments.
    - Evidence: file link.
11. **[M1-11] Dataset documentation**
    - Scope: `docs/dataset_documentation.md` covering the source and API version, collection method and scripts, collection dates, tags (including the AI contrast tag) and window, stratified sampling rule and weights, row counts, cross-check results, and limitations.
    - Licensing and ethics: content is CC BY-SA 4.0; outputs show titles only with links to the original questions and a licence notice; raw text is not published; user IDs are hashed.
    - This is the main evidence for the "dataset description" and "data source documented" rubric requirements.
    - Evidence: file link.

Phase 3: EDA support and reproducibility

12. **[M1-12] Dataset coverage charts**
    - Scope: two charts in `03_eda.ipynb` under a "Dataset coverage" heading: collected questions per tag per year, and collected versus total counts per month (showing the sampling rate), each with a written takeaway.
    - Evidence: notebook section.
13. **[M1-13] Re-runnable collection**

- Scope: settings (tags, dates, seed, filter) moved into a config file and a "how to re-run" section in the README, tested by re-running one tag-month from scratch.
- Evidence: pull request and README section.

Phase 4: Final notebook preparation

14. **[M1-14] `review1_main.ipynb` skeleton**
    - Scope: headings in rubric order (problem and objective, dataset and collection, cleaning, EDA, features, models, evaluation, business findings), with each section loading saved outputs or calling functions in `src/` rather than repeating code.
    - Evidence: notebook with headings and working loaders.

Phase 5: Integration and presentation

15. **[M1-15] Final notebook assembly and test run**
    - Scope: every member's final section combined, then a restart-and-run-all from a clean environment, with errors fixed or reported to the owner.
    - Evidence: executed notebook committed, with a comment confirming a clean run.
16. **[M1-16] Data collection slides and rehearsal**
    - Scope: two slides, one on the data source and collection method (with the log numbers) and one on the dataset overview (rows, fields, window, sampling). See Section 12.
    - Evidence: slides in the deck.

5. Member 2: data enrichment, cleaning and preprocessing

Member 2 completes the dataset by collecting answer and user details, then owns every cleaning decision, worth 3 marks. Each decision is justified in the notebook, not only applied.

Phase 0: Setup

1. **[M2-01] Shared Python environment**
    - Scope: agreed Python version and a `requirements.txt` with pinned versions of pandas, numpy, requests, python-dotenv, beautifulsoup4, pyarrow, scikit-learn, xgboost, lightgbm, shap, statsmodels, matplotlib, seaborn and spacy, tested with a clean install on local machines and Colab.
    - Evidence: file and a comment confirming a clean install. Complete when every member confirms it works.

2. **[M2-02] Enrichment design**
   - Scope: a design note based on the API docs for `/answers/{ids}`, `/questions/{ids}/answers` and `/users/{ids}`, covering three needs: the accepted answer's creation date, the first answer's creation date for every answered question (for the secondary target), and each asker's account creation date.
   - Also: batch sizes (100 IDs), fields, and the expected number of calls.
   - Evidence: design note in the issue.

Phase 1: Enrichment collection

3. **[M2-03] Enrichment script**
   - Scope: `src/enrich.py`, reusing Member 1's request function, with three batched steps: accepted answer IDs for accepted-answer dates; IDs of questions with at least one answer to `/questions/{ids}/answers`, keeping the earliest answer date per question; and owner user IDs for account creation dates.
   - Privacy: user IDs are hashed with SHA-256 and a secret salt from `.env` immediately after fetching; raw user IDs and names are never stored.
   - Every call is logged in the collection log.
   - Evidence: pull request. Complete when it works on Member 1's test pull.
4. **[M2-04] Enrichment run for all tags**
   - Scope: enrichment run tag by tag as question files arrive from Member 1, producing `answers.parquet` (accepted and first answer dates) and `users.parquet`.
   - Evidence: files, row counts and calls used, recorded in the issue.

Phase 2: Cleaning and preprocessing (lead phase)

5. **[M2-05] Merge and deduplication**
   - Scope: questions joined with answers and users. A question with two selected tags appears twice, so one row per `question_id` is kept with the full tag list and its `sampling_weight` (if the copies have different weights, the smaller is kept and noted).
   - Rows before and after, and duplicates removed, are recorded.
   - Evidence: `02_cleaning.ipynb` section.
6. **[M2-06] Timestamp conversion and checks**

- Scope: Unix times converted to UTC datetimes, with checks that no answer is earlier than its question and no date falls outside the window; any exceptions are investigated and documented.
- Evidence: notebook section with counts of problems found.

7. **[M2-07] HTML cleaning and code extraction**
   - Scope: code blocks (`pre` and `code`) extracted with BeautifulSoup into `code_text`, and `plain_body` produced without HTML or code.
   - Derived fields: `n_code_blocks`, `code_ratio`, and `has_error_text` (patterns such as Traceback, Exception, Error:, errno), with 3 before/after examples shown.
   - Evidence: notebook section and functions in `src/cleaning.py`.
8. **[M2-08] Missing values**
   - Scope: no accepted answer means target 0, not missing. Deleted asker accounts have no creation date, so the training-period median is imputed and a `missing_account_age` flag is added.
   - A missing-values table before and after, with the reason for each choice.
   - Evidence: notebook section.
9. **[M2-09] Both targets**
   - Main: `hours_to_accept` and `target_resolved_7d` (accepted answer within 168 hours). Secondary: `hours_to_first_answer` and `target_answered_7d` (any answer within 168 hours).
   - Label maturity: questions posted less than 30 days before the collection date are dropped so every label has matured, and the number dropped is recorded.
   - Also: `accepted_given_answered` (accepted within 7 days among questions answered within 7 days), to track whether acceptance habits changed over time.
   - Evidence: notebook section with class balance tables for both targets.
10. **[M2-10] Invalid and extreme records**
    - Scope: checks for empty bodies, extremely long bodies, and non-English or spam-like posts, with a justified keep, cap or remove decision for each. Rows are never removed based on the target.
    - Evidence: notebook section with decisions.
11. **[M2-11] Before/after cleaning summary**
    - Scope: one table covering rows, duplicates, missing values, invalid records, dropped immature labels, final rows, and class balance for both targets. This is the headline evidence for the cleaning marks.

- Evidence: table at the end of `02_cleaning.ipynb`.

12. **[M2-12] Processed dataset v1 and handoff**
    - Scope: `data/processed/questions_clean_v1.parquet` saved, with version and row count noted in the README, and handoff comments on Member 3′s and Member 4′s first Phase 3 tasks.
    - Evidence: file and handoff comments.

Phase 3: EDA support

13. **[M2-13] Aggregated tables for EDA**
    - Scope: monthly tables per tag using `sampling_weight`: resolution rate for both targets, median hours to accepted answer and to first answer, and `accepted_given_answered`. Weekly volume comes from Member 1′s complete counts, not from sampled records.
    - Tag-months with fewer than 30 sampled questions are flagged so charts can grey them out.
    - Output in `data/processed/`, used by Member 3 for trend charts and as the starting point for Review 2.
    - Evidence: files and a short check in the notebook.

Phase 4: Robustness

14. **[M2-14] Sampling robustness check**
    - Scope: sampled, weighted monthly volumes compared with the complete counts; weighted versus unweighted resolution rates; and class balance repeated with a second random seed on 2 tags.
    - Evidence: notebook section, with findings added to the limitations in the dataset documentation.

Phase 5: Integration and presentation

15. **[M2-15] Cleaning section for the main notebook**
    - Scope: `02_cleaning.ipynb` tidied and verified to run from raw files to v1, then handed to Member 1 for `review1_main.ipynb`.
    - Evidence: pull request.
16. **[M2-16] Cleaning slides and rehearsal**
    - Scope: two slides, one on key cleaning steps with before/after numbers and one on the targets and the label maturity rule. See Section 12.
    - Evidence: slides in the deck.

6. Member 3: problem framing, EDA and slide deck

Member 3 owns the story of Review 1: the problem statement, the EDA worth 3 marks, and the deck that ties all members' work together. Every chart answers a question and carries a written takeaway.

Phase 0: Setup

1. **[M3-01] Problem statement, objective and research questions**
   - Scope: the problem statement, business objective, stakeholder table and Review 1 research question, refined from the project plan. The stakeholder is framed broadly as any company running a developer or customer support forum. The draft is shared in the issue for team comments and saved as `docs/problem_statement.md`.
   - Evidence: file link. Complete when all members have commented or approved.
2. **[M3-02] Background sources and dated event list**
   - Scope: 3 to 5 credible sources on the Stack Overflow question decline and developer AI adoption, each recorded with title, link, date and the one fact used.
   - Also: `docs/event_list.csv` with dated events near the break, each with a source link: GitHub Copilot general availability (mid-2022), ChatGPT release (30 Nov 2022), Stack Overflow's ban on ChatGPT-generated answers (Dec 2022), and the moderator strike (mid-2023), plus any others found in research.
   - Evidence: `docs/background_sources.md` and the event list.

Phase 1: Planning

3. **[M3-03] EDA plan**
   - Scope: each EDA question with its chart type, data needed and owner (Member 1 handles the coverage charts), signed off by the team in the issue.
   - Evidence: plan in the issue or `docs/eda_plan.md`.
4. **[M3-04] Slide deck template**
   - Scope: the deck with the section order and slide owners from Section 12, a consistent theme, placeholder slides, and edit access for all members.
   - Evidence: deck link.

Phase 2: Early EDA

5. **[M3-05] First look at the test-pull data**

- Scope: distributions and obvious problems (odd dates, empty fields, very large bodies) in Member 1's test pull, with findings posted as comments on Member 2's cleaning tasks.
- Evidence: short notebook section and issue comments.

Phase 3: EDA (lead phase)

6. **[M3-06] Question volume trends**
   - Scope: weekly volume per tag from 2019 to 2026, from complete counts, with every event from the event list marked: one chart for the selected tags' total and one small-multiples chart by tag, on a log scale if tags differ greatly.
   - Also: the selected tags' share of the site-wide total over time, and the `openai-api` tag as a contrast line.
   - Takeaways describe timing ("the drop is consistent with…"), never cause.
   - Evidence: `03_eda.ipynb` section with takeaways.
7. **[M3-07] Target analysis**
   - Scope: class balance for both targets; monthly weighted resolution rates for both targets by tag, from Member 2's tables, with low-sample months greyed out; and a pre- versus post-November 2022 comparison table.
   - Also: `accepted_given_answered` over time. A fall would mean acceptance habits changed, which affects how the main target is interpreted.
   - Evidence: notebook section with takeaways.
8. **[M3-08] Time to answer**
   - Scope: distributions of hours to accepted answer and to first answer (log scale), overall and by tag, and monthly weighted medians before and after November 2022.
   - Evidence: notebook section with takeaways.
9. **[M3-09] Question characteristics and resolution**
   - Scope: resolution rate by code present, error text present, body length band, number of tags, and question word, as bar charts with the count shown on each bar.
   - Evidence: notebook section with takeaways.
10. **[M3-10] Asker and timing patterns**
    - Scope: resolution rate by account age band and by new account (created within 7 days of posting), and an hour-by-weekday heatmap (UTC), weighted so weekdays and weekend days are represented correctly.
    - Evidence: notebook section with takeaways.

11. **[M3-11] Statistical checks**
    - Scope: chi-square tests for categorical features against the target, Mann-Whitney tests for length features, and a correlation heatmap of numeric features, reporting effect sizes as well as p-values because the sample is large.
    - Evidence: notebook section.
12. **[M3-12] EDA summary and feature recommendations**
    - Scope: a top-5 insights list at the end of the EDA notebook, each linked to a feature for Member 4 to build or check, posted as a comment on Member 4′s feature tasks.
    - Evidence: notebook section and handoff comment.

Phase 4: Business framing

13. **[M3-13] Initial business findings page (with Member 5)**
    - Scope: a one-page findings document with 3 to 5 findings from EDA and the model, each with a number, the stakeholder it matters to, and a suggested action. Member 3 writes the EDA findings and edits the final page.
    - Wording: "associated with" or "consistent with" for anything related to AI; example questions appear as titles with links only.
    - Evidence: `reports/review1/initial_findings.md`.
14. **[M3-14] Deck completion**
    - Scope: all members′ slides collected, with consistent chart style and wording, a takeaway in every chart slide′s title, and every headline number matching Member 1′s verified figures.
    - Evidence: deck version with all sections filled.

Phase 5: Presentation

15. **[M3-15] Rehearsals**
    - Scope: two timed full run-throughs against the 10-minute limit, with overruns noted per speaker and slides adjusted, and likely viva questions from all members collected into one list.
    - Evidence: rehearsal notes in the issue.
16. **[M3-16] Problem and EDA slides**
    - Scope: the opening (problem, objective, why it matters) and the EDA highlights. See Section 12.
    - Evidence: slides in the deck.

7. Member 4: features, dimensionality reduction and baseline models

Member 4 turns clean data into model-ready features without leakage, owns the feature engineering marks, and builds the split and baseline models that Member 5′s models must beat.

Phase 0: Setup

1. **[M4-01] Split and validation strategy**
   - Scope: the chronological split (train Jan 2019 to Dec 2024, validation Jan to Jun 2025, test Jul 2025 to Jul 2026), the time-series cross-validation folds inside the training period, and the reason a random split would leak future patterns.
   - Also: the rolling-origin evaluation, designed with Member 5: training on all data before each half-year and testing on that half-year, from 2021 H1 to 2026 H1. This gives results both before and after November 2022, which the single test period cannot.
   - Evidence: `docs/split_strategy.md`.
2. **[M4-02] Feature specification**
   - Scope: a table of every candidate feature with its name, formula, source field, and whether it is available at posting time, with every forbidden field from the leakage rule marked.
   - Excluded by design: `post_ai` (constant in the test period, used only for analysis) and `first_question` (unreliable with sampled data). Replacements: `new_account` (account created within 7 days of posting) and lagged tag volume (task M4-06).
   - Evidence: `docs/feature_spec.md`, reviewed by Member 5.

Phase 1: Feature code on sample data

3. **[M4-03] Feature functions**
   - Scope: functions in `src/features.py` for each structured feature, built on Member 1′s test pull, with small tests on 5 hand-checked rows.
   - Evidence: pull request with tests passing.

Phase 2: Split module

4. **[M4-04] Split module and verification**

- Scope: `src/split.py` splitting by `creation_date` and creating the time-series folds, with an assertion that no question ID appears in two splits, and a report of rows and class balance per split.
- Evidence: pull request and a table in `04_features.ipynb`.

Phase 3: Feature engineering (lead phase)

5. **[M4-05] Structured features on the full data**
   - Scope: title and body lengths, code features, error text, question word, number of tags, hour, weekday, month, account age, `new_account`, and the missing account age flag, plus Member 3's EDA recommendations.
   - Evidence: notebook section with summary statistics per feature.
6. **[M4-06] Tag encoding and lagged tag volume**
   - Scope: multi-hot columns for the selected tags plus an "other tags count".
   - Lagged volume from Member 1's complete weekly counts: for each question, the total questions in its tags during the 4 full weeks before its posting week, and the change versus the 4 weeks before that. Weeks on or after the posting date are never used.
   - Evidence: notebook section with a check on 3 hand-picked questions.
7. **[M4-07] Text features with dimensionality reduction**
   - Scope: TF-IDF on titles (unigrams and bigrams, up to 5,000 terms), fitted on training data only, reduced with TruncatedSVD, with a cumulative explained variance plot and a stated reason for the number of components (around 100).
   - Evidence: notebook section with the explained variance plot. This covers the dimensionality reduction part of the rubric.
8. **[M4-08] Leakage review**
   - Scope: every column in the final feature matrix checked against the feature spec: no forbidden field, no `post_ai`, no `first_question`, and lagged volume using only past weeks. Member 5 double-checks and signs off in the issue.
   - Evidence: checklist in the notebook and sign-off comment.
9. **[M4-09] Preprocessing pipeline**
   - Scope: a scikit-learn `ColumnTransformer` and `Pipeline` that scales numeric features, passes binary ones, and applies TF-IDF and SVD, all fitted on training data only, saved with joblib and able to refit cleanly inside each rolling-origin fold.
   - Evidence: pull request and saved pipeline file.

Phase 4: Baseline models

10. **[M4-10] Baseline models**
    - Scope: a majority-class baseline and logistic regression with class weights, evaluated on validation with Member 5′s evaluation functions (precision, recall, F1, PR-AUC, ROC-AUC).
    - Evidence: `05_models.ipynb` section with a results table.
11. **[M4-11] Feature gain experiment**
    - Scope: logistic regression on structured features only versus structured plus text features, showing the difference in F1 and PR-AUC. This demonstrates that the engineered features help.
    - Evidence: comparison table and a one-line takeaway.
12. **[M4-12] Interpretable coefficients**
    - Scope: the top 15 positive and negative logistic regression coefficients, each explained in plain words (for example, "questions with error text are more likely to be resolved").
    - Evidence: notebook section.
13. **[M4-13] Handoff to Member 5**
    - Scope: feature matrices and pipeline saved, loading instructions documented, and a comment on Member 5′s modelling tasks confirming they are ready.
    - Evidence: handoff comment.

Phase 5: Integration and presentation

14. **[M4-14] Feature and baseline sections for the main notebook**
    - Scope: `04_features.ipynb` and the baseline part of `05_models.ipynb` tidied and handed to Member 1.
    - Evidence: pull request.
15. **[M4-15] Features and baseline slides**
    - Scope: two slides, one on feature engineering and dimensionality reduction (with the feature gain result) and one on the split and baseline results. See Section 12.
    - Evidence: slides in the deck.

8. Member 5: advanced models, evaluation and business findings

Member 5 builds the strongest model, owns evaluation and interpretation worth 3 marks, and turns the results into business findings. Early phases prepare the evaluation tools so modelling can start the moment features are ready.

Phase 0: Setup

1. **[M5-01] Evaluation plan**
   - Scope: the metrics (precision, recall, F1, PR-AUC, ROC-AUC, confusion matrix), the reason accuracy is never used alone, and the business metric: the share of unresolved questions captured by flagging the 20% of questions with the lowest predicted resolution.
   - Rules: the decision threshold is tuned on the validation period only, and the test period is used once, after all tuning.
   - Also: the rolling-origin evaluation designed with Member 4, and the secondary-target check.
   - Evidence: `docs/evaluation_plan.md`.
2. **[M5-02] Modelling skeleton**
   - Scope: Random Forest and XGBoost or LightGBM inside the same scikit-learn pipeline shape Member 4 uses, with `RandomizedSearchCV` and time-series folds, tested on a small synthetic dataset.
   - Evidence: pull request.

Phase 1: Evaluation tools

3. **[M5-03] Evaluation module**
   - Scope: `src/evaluate.py` with a metrics table function, confusion matrix plot, precision-recall and ROC curves, and the capture-rate business metric, tested on toy data, with usage notes shared with Member 4.
   - Evidence: pull request and example outputs.

Phase 2: Error analysis design

4. **[M5-04] Error analysis design**
   - Scope: the method for sampling 30 confident wrong predictions (15 false positives, 15 false negatives) and the annotation sheet columns: title, tags, predicted probability, actual outcome, likely reason.
   - Evidence: template in the issue or `docs/`.

Phase 3: Checks and smoke test

5. **[M5-05] Feature spec and leakage review**
   - Scope: review of Member 4's feature spec and leakage checklist, with sign-off or a list of problems in the issue.

- Evidence: sign-off comment.

6. **[M5-06] Modelling pipeline smoke test**
   - Scope: the skeleton run on an early version of the feature matrix to find integration problems before full training.
   - Evidence: comment listing issues found and fixed.

Phase 4: Models and evaluation (lead phase)

7. **[M5-07] Random Forest**
   - Scope: training with class weights, tuning the number of trees, depth and minimum leaf size with time-series folds, and recording the best parameters and validation metrics.
   - Evidence: `05_models.ipynb` section.
8. **[M5-08] Gradient boosting**
   - Scope: XGBoost or LightGBM with `scale_pos_weight` for imbalance, tuning learning rate, depth and number of trees, with early stopping on validation.
   - Evidence: `05_models.ipynb` section.
9. **[M5-09] Model comparison and threshold**
   - Scope: one validation table covering the majority baseline, logistic regression (from Member 4), Random Forest and boosting; the best model chosen with a written reason; and the decision threshold tuned on validation.
   - Evidence: comparison table and reasoning.
10. **[M5-10] Final test evaluation**
    - Scope: a single evaluation of the chosen model on the test period, with metrics table, confusion matrix, precision-recall and ROC curves, capture-rate result, and test set size. No tuning happens after this step.
    - Evidence: `06_evaluation.ipynb` section.
11. **[M5-11] SHAP interpretation**
    - Scope: a SHAP summary plot and dependence plots for the top 5 features, each explained in one plain sentence, compared with Member 4′s logistic regression coefficients.
    - Evidence: notebook section.
12. **[M5-12] Rolling-origin evaluation and pre/post-AI comparison**

- Scope: the chosen model settings trained on all data before each half-year and tested on that half-year, from 2021 H1 to 2026 H1, with PR-AUC and F1 plotted per half-year alongside the November 2022 marker, and top SHAP features compared between one pre-AI and one post-AI window.
- Evidence: notebook section with the chart and a written interpretation.

13. **[M5-13] Secondary target check**
    - Scope: the chosen model retrained on `target_answered_7d`, with metrics and top features compared against the main target. Any difference in conclusions is explained (for example, changed acceptance habits shown in Member 3's EDA).
    - Evidence: comparison table in `06_evaluation.ipynb`.
14. **[M5-14] Error analysis**
    - Scope: the annotation sheet completed for 30 confident errors, with the patterns summarised (for example, very short titles or rare tags).
    - Evidence: table and summary in the notebook.
15. **[M5-15] Initial business findings (with Member 3)**
    - Scope: the model findings for the shared findings page: what the model offers a support-forum manager, with the capture-rate number and its limits (acceptance as a proxy, smaller recent data).
    - Evidence: section in `reports/review1/initial_findings.md`.

Phase 5: Integration and presentation

16. **[M5-16] Model and evaluation sections for the main notebook**
    - Scope: the advanced part of `05_models.ipynb` and all of `06_evaluation.ipynb` tidied, the final model saved with joblib, and the sections handed to Member 1.
    - Evidence: pull request.
17. **[M5-17] Models, evaluation and findings slides**
    - Scope: two to three slides covering model comparison, evaluation with SHAP and rolling-origin results, and business findings. See Section 12.
    - Evidence: slides in the deck.

9. Shared tasks for every member

These tasks appear on the board separately for each member (`X` stands for the member number), so every member's participation shows in the board history.

Phase	GitHub issue title	Description	Evidence
Phase 0	[MX-00] Rubric and project plan reading	The rubric PDF and project plan are read, and one comment lists the member's rubric components and any questions	Issue comment
Every phase	[MX-MEET] Team check-in notes (one shared issue, rotating note-taker)	Progress, blockers and next tasks are posted at each check-in; the note-taker records decisions	Comments on the shared issue
Every phase	Partner task reviews (no separate issue)	When a partner's task reaches Review/Testing, the reviewer checks the evidence, runs the code where relevant, comments on what was verified, then approves	Review comments on their issues and pull requests
Phase 4	[MX-VIVA] Viva questions for the member's area	5 likely examiner questions about the member's area, with short answers, added to the shared list kept by Member 3	Comment on Member 3's rehearsal issue
Phase 5	[MX-LEARN] Whole-pipeline understanding	Every other member's notebook section is read and can be explained in two sentences	Comment summarising what was checked
Phase 5	[MX-SUM] Individual contribution summary	The template in Section 13 is filled, linking the member's issues, pull requests and slides	Summary file in `reports/review1/`

Phase	GitHub issue title	Description	Evidence
Phase 5	[MX-FINAL] Board clean-up	Every task owned by the member is Completed, with evidence, an assignee and a due date	Board view filtered to the member

Rules that apply to everyone

- Work happens only on tasks that exist on the board; unplanned work gets a task first.
- Notebooks stay runnable: restart and run all happens before any notebook task moves to Review/Testing.
- Help requested and help given go in task comments, not only in chat, so both are visible.
- Delays are raised early in the check-in issue so tasks can be reassigned with a visible record.

10. Peer review pairs and handoffs

Each member reviews exactly one other member's tasks, chosen so the reviewer uses that work next, and every handoff is announced as a comment on the receiving member's task.

Review pairs (set in each task's Reviewer field)

Owner	Reviewer	Why this pairing
Member 1	Member 4	Member 4's features depend on the collected fields and data dictionary
Member 2	Member 1	Member 1 knows the raw data best
Member 3	Member 2	EDA uses Member 2's cleaned data and monthly tables
Member 4	Member 5	Member 5 must trust the features and leakage check
Member 5	Member 3	Member 3 turns model results into the business story

Handoffs

From	To	What is handed over	When
Member 1	Members 3 and 4	Test-pull sample of python questions	Phase 1, after [M1-05]
Member 5	Member 4	Evaluation module and usage notes	Phase 1, after [M5-03]
Member 1	Member 2	Question files, tag by tag	Phase 1, during [M1-07]
Member 2	Members 3 and 4	`questions_clean_v1.parquet`	End of Phase 2, [M2-12]
Member 2	Member 3	Monthly weighted resolution and time-to-answer tables	Phase 3, [M2-13]
Member 3	Member 4	Top-5 EDA insights and feature suggestions	Phase 3, [M3-12]
Member 4	Member 5	Feature matrices, saved pipeline, leakage sign-off	End of Phase 3 and [M4-13]
Member 5	Member 3	Model results and findings text	Phase 4, [M5-15]
All members	Member 1	Final notebook sections	Phase 5
All members	Member 3	Finished slides	Phase 5

The handoff comment should say what was delivered, where it is, the row count or version, and any known issues.

11. Review 1 deliverables and rubric checklist

Before the review, every row below must be ticked with evidence; the rubric lists four deliverables plus the board link and contribution summaries.

Rubric deliverables

- [ ] Jupyter Notebook: `review1_main.ipynb`, runs top to bottom (Member 1, with sections from all)

- [ ] Dataset documentation: `docs/dataset_documentation.md` and `docs/data_dictionary.md` (Member 1)
- [ ] Preprocessing, exploratory analysis and visualisations: cleaning and EDA sections (Members 2 and 3)
- [ ] Predictive model, evaluation and initial business findings: model sections and `initial_findings.md` (Members 4, 5 and 3)
- [ ] 10-minute team presentation (all, deck by Member 3)
- [ ] Project board link (Member 1)
- [ ] Individual contribution summary for each member (all)

Rubric component check

Component	Marks	Evidence that must exist	Owner	Ready
Problem statement, objective, dataset description	2	Problem statement doc, dataset documentation, data dictionary	Members 3 and 1	
Data cleaning and preprocessing	3	Before/after table, justified decisions, both targets, sampling weights, maturity rule	Member 2	
EDA and visualisation	3	10 to 12 charts, each with a takeaway, statistical checks, top-5 insights	Member 3 (coverage charts by Member 1)	
Feature engineering or dimensionality reduction	2	Feature spec, TF-IDF + SVD with variance plot, feature gain table	Member 4	
Predictive model development	4	Baselines, Random Forest, boosting, tuning with time-series folds, comparison table	Members 4 and 5	
Model evaluation and interpretation	3	Test metrics, curves, SHAP, rolling-origin results, secondary-target check, error analysis, business translation	Member 5	

Component	Marks	Evidence that must exist	Owner	Ready
Presentation and individual contribution	3	Rehearsed deck, board history, contribution summaries	All	

Quality checks before the review

- [ ] No leakage fields in the final feature matrix, including no `post_ai` or `first_question` (Member 4 checklist signed by Member 5)
- [ ] Lagged tag volume uses only weeks before each question
- [ ] Test set used only once, after all tuning; threshold tuned on validation
- [ ] Rolling-origin results and secondary-target check included
- [ ] Monthly resolution figures use sampling weights; low-sample months flagged
- [ ] Every chart has a title, axis labels, units and a written takeaway
- [ ] Every cleaning decision has a one-line reason
- [ ] Headline numbers verified in Data Explorer and matching across notebook and slides
- [ ] AI-related claims say "associated with" or "consistent with", never "caused"
- [ ] Licence notice (CC BY-SA 4.0) and title-only examples with links
- [ ] Every board task has an assignee, due date, evidence link and review comment
- [ ] API key and raw user IDs are not in the repository

12. 10-minute presentation plan

Every member speaks for 1.5 to 2.5 minutes about the work they led, in the order the data flows, so examiners can see each person's contribution.

Order	Speaker	Slides	Minutes	What to show
1	Member 3	Title; problem and objective	1.0	The decline in numbers, the stakeholder, the Review 1 question
2	Member 1	Data collection; dataset overview	1.5	API, sampling rule, calls and rows from the log, cross-check result

Order	Speaker	Slides	Minutes	What to show
3	Member 2	Cleaning steps; target and label maturity	1.5	Before/after table, HTML and code extraction, both 7-day targets and sampling weights
4	Member 3	EDA highlights (2 slides)	1.5	3 strongest charts, each with its takeaway
5	Member 4	Features and dimensionality reduction; split and baselines	2.0	Feature gain table, SVD variance plot, chronological split
6	Member 5	Model comparison; evaluation and SHAP; business findings	2.5	Comparison table, test metrics, rolling-origin chart, top SHAP features, capture-rate finding
Total		11 to 12 slides	10.0	

Rules for the deck

- Slide titles state the finding ("Questions with code are resolved more often"), not the topic ("Code analysis").
- One main chart per slide, large enough to read from the back of the room.
- Numbers on slides match the notebook exactly.
- The final slide carries the board link and the repository link.

During questions: the member who led the area answers first; others add only if asked. Everyone must be able to answer a basic question about any part of the pipeline.

13. Individual contribution summary template

Each member submits one page using this template, saved as `reports/review1/contribution_memberX.md`, with every claim linked to board evidence.

```
# Review 1 Contribution Summary: Member X (Name, Register number)

Role: <lead area>
Rubric components owned: <components and marks>
```

```
## Tasks completed
| Issue | Title | Evidence link | Reviewed by |
| --- | --- | --- | --- |
| #12 | [MX-05] ... | <PR or notebook link> | Member Y |

## Key outputs
- <file, notebook section, chart or slide, with link>

## Decisions I made and why
- <decision>: <one-line reason>

## Reviews I gave
- <issue numbers of partner tasks I reviewed>

## Problems faced and how I solved them
- <problem>: <solution>

## What I can explain in the viva
- <3 to 5 points about my area>
- <1 to 2 points about another member's area>
```

14. Review 1 viva answers

Every member should be able to give these answers in their own words; the member who owns the area answers first.

Likely question	Model answer	Owner
Is an accepted answer within 7 days the same as resolved?	No, acceptance is a strict proxy that depends on the asker returning. We also report any answer within 7 days and track acceptance among answered questions over time.	Members 2 and 5
Your data is sampled before 2025. How do you know trends are not distorted?	We sample weekdays and weekend days separately, weight them back to each month's real mix, compare weighted volumes with complete counts, and repeat with a second seed.	Members 1 and 2

Likely question	Model answer	Owner
Why is there no post-AI feature in the model?	It would be constant in the test period, so the model could not use it there. We use the period only for analysis and rolling-origin comparisons.	Member 4
How did you tune the threshold without touching the test set?	On the validation period only; the test period was evaluated once, afterwards.	Member 5
How can you compare pre- and post-AI performance if the test set is all post-AI?	Rolling-origin evaluation: we train on everything before each half-year and test on it, from 2021 to 2026.	Member 5
Why these tags? Do they represent the whole site?	They are popular and varied, with one AI tag for contrast. We show their share of all questions and do not generalise beyond them.	Member 1
How could tag popularity leak future information?	Averaging over the whole period would. We use only the 4 weeks before each question's posting week.	Member 4
Did AI cause the drop?	We cannot prove cause. The drop is consistent with AI adoption, but other events such as the ChatGPT-answer ban and the moderator strike happened close by.	Member 3
How would a manager act on the resolution score?	Route low-probability questions to experts, or prompt askers to add code or error text before posting.	Member 5
Walk us through your own code.	Each member explains every function in their own notebook section, what it does, and why each design choice was made.	Everyone